

System Design — Day 3

Topic: Caching — CDN, Redis/Memcached, Cache Invalidation

1. What Is a Cache?

A cache is a fast, temporary storage layer that holds the result of an expensive operation, so repeated requests get served instantly instead of redoing the work.

Example: instead of recalculating 'trending photos' from the database on every one of 1 million requests/minute, compute it once, store the result in Redis, and serve every request from that cached copy.

2. TTL (Time To Live) / Cache Expiration

A cached value is set to auto-expire after a fixed time (e.g. 5 minutes for a trending list), forcing a fresh recalculation only periodically instead of on every request.

3. Cache Stampede (Thundering Herd)

What happens: the moment a cache expires, thousands of simultaneous requests can all see an empty cache at once. With no coordination between them, each one independently decides to run the expensive database query itself.

Why it's dangerous: the database — previously protected by the cache absorbing almost all traffic — suddenly gets slammed with thousands of simultaneous expensive queries at once, which can overload or crash it.

Common fix: use a lock so only one request recomputes the value while everyone else waits briefly or is served slightly-stale data.

4. CDN (Content Delivery Network)

Copies of static content (images, videos, CSS/JS files) are stored on servers physically distributed around the world.

A user's request is routed to the nearest copy instead of traveling all the way to the origin server.

Good fit: public, static, frequently-requested content (a public video, a product image, a JS bundle).

Bad fit: private, sensitive, or constantly-changing data (private messages) — you don't want sensitive data distributed across edge servers worldwide, and there's nothing to 'cache' if it's unique per user and rarely repeated.

5. Redis vs CDN — Different Problems, Not Interchangeable

	Redis (application cache)	CDN
Problem solved	Avoiding repeated expensive computation / database load	Avoiding slow network travel distance to the user
Located	Near your servers/database	Distributed globally, near the user
Example	Cached 'top 20 trending photos' list	Cached copy of an actual image/video file

6. Key Takeaway

Caching (Redis) and CDNs both make systems faster, but they solve fundamentally different problems — one avoids redundant computation, the other avoids network distance. A well-designed system often uses both for the same content, for different reasons. Always watch for the cache stampede failure mode when a cache expires under heavy load.